Check for updates

**Glen Williams**
Mem. ASME
Mechanical Engineering,
The Pennsylvania State University,
137 Reber Building,
University Park, PA 16802-4400
e-mail: gtw5020@psu.edu

**Nicholas A. Meisel**
Mem. ASME
Engineering Design,
The Pennsylvania State University,
213 Hammond Building,
University Park, PA 16802
e-mail: nam20@psu.edu

**Timothy W. Simpson**
Fellow ASME
Mechanical Engineering,
The Pennsylvania State University,
137 Reber Building,
University Park, PA 16802-4400
e-mail: tws8@psu.edu

**Christopher McComb**[1]
Mem. ASME
Engineering Design,
The Pennsylvania State University,
213 Hammond Building,
University Park, PA 16802
e-mail: mccomb@psu.edu

# Design Repository Effectiveness for 3D Convolutional Neural Networks: Application to Additive Manufacturing

*Machine learning can be used to automate common or time-consuming engineering tasks for which sufficient data already exist. For instance, design repositories can be used to train deep learning algorithms to assess component manufacturability; however, methods to determine the suitability of a design repository for use with machine learning do not exist. We provide an initial investigation toward identifying such a method using "artificial" design repositories to experimentally test the extent to which altering properties of the dataset impacts the assessment precision and generalizability of neural networks trained on the data. For this experiment, we use a 3D convolutional neural network to estimate quantitative manufacturing metrics directly from voxel-based component geometries. Additive manufacturing (AM) is used as a case study because of the recent growth of AM-focused design repositories such as GrabCAD and Thingiverse that are readily accessible online. In this study, we focus only on material extrusion, the dominant consumer AM process, and investigate three AM build metrics: (1) part mass, (2) support material mass, and (3) build time. Additionally, we compare the convolutional neural network accuracy to that of a baseline multiple linear regression model. Our results suggest that training on design repositories with less standardized orientation and position resulted in more accurate trained neural networks and that orientation-dependent metrics were harder to estimate than orientation-independent metrics. Furthermore, the convolutional neural network was more accurate than the baseline linear regression model for all build metrics.*
[DOI: 10.1115/1.4044199]

*Keywords: design automation, design evaluation, design for manufacturing*

## 1 Introduction

Machine learning (ML) presents an exciting opportunity to make use of the ever-growing supply of accumulated digital data available to engineers. The ability of some ML algorithms, such as deep neural networks [1], to intake a wide variety of data formats and output equally diverse, useful metrics makes them advantageous for applications in many industries. Digitally driven manufacturing is an example of one such industry that accrues massive amounts of digital data simply as a by-product [2]. In this digitally driven domain, designers can conceptualize, assess, and refine a design in an entirely virtual environment, resulting in the creation of high-quality, digital records that are often compiled and stored in design databases and repositories [3]. Examples can be found in consumer-oriented additive manufacturing (AM), which has led to the proliferation of online, crowd-sourced repositories such as GrabCAD[2] and Thingiverse.[3] Designers can use these websites to collectively share hundreds of thousands of digital designs.

In addition to serving as valuable archives and collaboration tools, design repositories can benefit the technical design process by being used in conjunction with ML software tools to automate geometry-specific analysis. For instance, ML algorithms can use prior design data to analyze functionality and manufacturing metrics before a design is ever fabricated [4–8]. Deploying ML constructs in AM applications could be particularly useful because the flexibility and reusability of many ML analytical techniques [1] could meet the challengingly high variability present in AM designs, which take advantage of many freedoms of complexity beyond traditional manufacturing [9]. This is made possible through design repositories which offer rich 3D computer-aided design (CAD) data that may be processed by ML constructs, such as 3D convolutional neural networks (CNN), to assess design attributes directly from a CAD model of the component geometry [4,5,10].

Although design repositories are growing in size and number, determining whether a repository is useful for training effective ML tools remains a largely subjective task. Typically, ML approaches require large and high-quality datasets [1]. Knowing what attributes of a repository make it an advantageous or disadvantageous training dataset could add value to the industry by helping designers contribute to or curate repositories more meaningfully. More informed contributions could empower designers to have a more useful long-term impact on the community. Through experimental study of design repository attributes themselves, greater quantitative meaning and understanding regarding repository effectiveness could be uncovered, expanding beyond subjective heuristics that are often employed today.

The purpose of this study is to experimentally investigate the extent to which enforcing design repository standardization (specifically with respect to positioning and orientation) impacts the capability of an ML construct to analyze new geometric data. To enable this experiment, we implement a 3D CNN that assesses build metrics of part mass, required support material mass, and build time for creating a part with material extrusion AM. The inputs to the CNNs were binary voxelized AM part design geometries and

[1]Corresponding author.
[2]https://grabcad.com/
[3]https://www.thingiverse.com

Contributed by the Design for Manufacturing Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received March 1, 2019; final manuscript received June 27, 2019; published online July 9, 2019. Assoc. Editor: Yaoyao Fiona Zhao.

 Copyright © 2019 by ASME 

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

the outputs were associated build metric predictions. CNNs were trained and used to experimentally evaluate designs from different artificial repositories to explore the following questions:

(1) What is the effect of design repository standardization on CNN accuracy?
(2) What is the effect of orientation dependence of the value being modeled on CNN accuracy?
(3) How does CNN accuracy differ from a baseline linear regression model?

The value of studying these three research questions is identifying how design repository standardization actions, such as enforcing orientation and translation consistency, impact the performance of CNNs used for the analysis of AM build metrics. These metrics are often nonintuitive for humans to predict, so an ML-based solution carries significant value. This work exists at the intersection between three subdisciplines: convolutional neural networks for 3D data, design repository research, and machine learning in design for AM. In Secs. 1.1–1.3, we synthesize relevant literature from all three of these areas in order to appropriately identify the motivation for this interdisciplinary study.

**1.1 Convolutional Neural Networks for 3D Data.** Artificial neural networks (ANNs) map high-dimensional inputs to outputs in a versatile and reusable way by learning from labeled datasets [11]. Although ANNs and the data that train them may take many forms, CNNs are a particularly beneficial type of ANN for 2D and 3D data because they efficiently scale with higher input resolutions and can interpret spatial features, such as shapes and edges [1]. These feature-finding tools can be implemented for 1D, 2D, 3D, or even higher-dimensional data formats. The image processing industry depends on digital media [12], and for that reason, many researchers have used image repositories to pioneer CNN image recognition techniques [13]. Researchers have also exploited parallels between 2D images and 3D geometric data to create analogous ML techniques for product design, robotic vision, and medical imaging applications [4,7–10,14]. Some researchers from these disciplines used voxel-based inputs to CNNs. Voxel-based models represent volumetric occupancy throughout a discretized 3D space. They identify which volume sections are populated with material and which are empty and have the advantage of easy integration into 2D and 3D CNNs, which require fixed-space inputs [4].

Despite the diversity of contemporary ML with 3D input (3D ML) research topics, most 3D ML research shares commonalities and can be categorized based on two dominant goals: (1) classification and (2) regression. In classification problems, 3D geometries are either grouped into categories or assigned discretized attribute levels based on inputs to an ANN [11]. Maturana and Scherer used a 3D CNN for object recognition via categorization in robotic vision applications [10]. However, not all classification problems are categorical. For example, Dering and Tucker used CNN classification to predict whether or not an object could perform certain functions, resulting in products being classified by more than one attribute [4]. Other problems require voxel-by-voxel classification solutions, such as a 3D CNN developed to separate magnetic resonance imaging scans of head matter into skull and nonskull regions for diagnostic purposes [15].

Although classification has many useful applications, some metrics, like the degree of part warping in AM, are numeric and continuous in nature [16]. Such applications require regression ANNs, which map inputs to continuous estimated values [17]. Compared to classification, relatively few regression 3D ANN studies have been completed. This gap presents opportunities to investigate how well quantitative metrics may be predicted for 3D geometric ML regression applications. Furthermore, one distinguishing difference between voxel-based geometric ML applications and most other machine learning fields is the dimensionality or number of independent variables of a single sample. Voxel-based models can have extremely high dimensionality, numbering in the tens or hundreds of thousands of independent variables [4]. Although high-dimensional problems are not unique to voxel-based applications, they pose challenges for some regression methods, such as linear regression techniques, for which heuristics have been developed relating the sample size to the recommended number of independent variables [18]. The authors are not aware of research that directly compared tools commonly used for high-dimensional regression analysis, such as artificial neural networks, to those that are generally more common and simple to implement, such as linear regressions, in voxel-based geometric regression applications.

**1.2 Design Repository Research.** Before 3D ML constructs can be deployed in the real world, suitable datasets containing applicable 3D models must be established. Artificial neural network performance can be very sensitive to size and variation of the training data, making it necessary for ML developers to clearly understand the data that they use [1]. Researchers have been investigating how to create and maintain design repository datasets for decades and have showcased many ways they can benefit designers. In 1997, Regli and Gaines published an online design repository for 2D and 3D standard CAD format parts on behalf of the National Institute of Standards and Technology (NIST) [19]. Their intent was to pioneer design repository practices for use in future commercial software [19]. Szykman [20] extended this research with a more complete description of the NIST Design Repository Project's online database management system beyond the initial user interface. Efforts have since been made to expand on the initial types of information stored in design repositories and the usefulness of that information. Bohm et al. [21] demonstrated techniques for efficiently storing design information to maximize potential for design reuse. Additionally, Bohm et al. [22] proposed a more complete, standardized approach to expand connections between artifacts, functions, failures, physical attributes, performance, sensory attributes, and available media. Researchers have also shown that design repositories can improve the design process itself. For instance, searching a design repository for required product functions can augment the conceptualization process [23] or be used as a replacement for physical product dissection activities [24].

Although guidelines for implementing real-world design repositories have been proposed, few 3D ML studies focus on the characteristics of the data repository used to train the ML construct. On the contrary, many 3D geometry ANN studies compare the performance of a new ANN architecture to other networks from similar applications using benchmark data [4,10]. Although this is an important exercise in advancing ML, it is less impactful in demonstrating the fundamental usefulness of ML to real-world geometric repositories that may have attributes that are not applicable to the available benchmark datasets. This challenge is evident from the commonalities of digital design repositories that have been published as benchmarking datasets, many of which are classification-focused. Two popular datasets are ModelNet [25] and ShapeNet [26]. These datasets contain 3D geometries organized into categories and labeled with various annotations [25,26]. While this type of repository is well suited for its primary goal of supporting classification research, there is no guarantee that the geometries are suitable for use in application-driven regression ANNs, which require application-relevant geometries and associated quantitative training labels.

Other researchers have used different types of design repositories or case studies for input data to 3D ML. These have included manually designed case studies [7]; designs pulled randomly from online, crowd-sourced repositories [8]; and design competition entrants [5]. Although these sources do offer alternatives to standard benchmarking repositories that might be more appropriate to specific applications, they often result in substantially lower sample sizes. Examples in the literature typically use less than 1000 unique designs [5,7,8]. These sample sizes are at least two orders of magnitude smaller than many realistic design repositories [2],



Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

and they likely present a limitation when attempting to transition ML constructs from research to industrial use in the future.

Additionally, the relative lack of design repositories makes assessing the quality of the datasets for ML training a challenge. To foster the growth of improved design repositories, greater understanding regarding the effects of ML training dataset attributes is needed. This knowledge could help inform design repository curators and contributors how to better control their design processes and increase the usefulness of design repositories for ML training in the future. Achieving more efficient and informed repository control on an ongoing basis could both reduce the cost of preparing data for ML training and maximize the likelihood of extracting usable knowledge from that data. Although some ML studies do develop theory around datasets in general, more targeted research regarding how application-specific dataset attributes affect ML training performance is needed to translate broader ML theories into actionable improvements to design repository curation techniques. Broadening experimental techniques to examine the attributes of datasets in new ways could uncover the basis for these improvements.

**1.3 Application of Machine Learning to Design for Additive Manufacturing.** AM is one area of 3D geometric research that is likely to benefit from increased ML research. Researchers have applied ML to AM in many ways, demonstrating potential improvements to diverse applications including process optimization [27–33], manufacturing defect detection and control [34–41], cybersecurity [42–44], dimensional accuracy analysis [9,45–47], and material property prediction [48–51]. The majority of the extensive body of literature representing the combination of AM and ML focuses on specific manufacturing processes and materials. The current work, in contrast, is primarily focused on the application of ML to the process of designing objects to be manufactured with those techniques, a domain best known as design for additive manufacturing (DfAM). Components designed for and manufactured using AM are passed through a digital thread of file formats before being additively manufactured, and the files are often stored in digital repositories afterward [52]. During the design process, AM designers are challenged to consider each design's build metrics, geometric aspects of a component that affect how easily or costly it may be additively manufactured. They must often do this while navigating some or all of the seven distinct AM process types: (i) material extrusion, (ii) powder-bed fusion, (iii) material jetting, (iv) binder jetting, (v) directed energy deposition, (vi) vat photopolymerization, and (vii) sheet lamination [53]. Currently, the sheer number of and differences between these AM processes can make manually estimating build metrics challenging [54]. Heuristics can accelerate the ability of designers to estimate AM build metrics [55], but companies and individuals must invest time and money to learn appropriate heuristics. Furthermore, the heuristics can even vary depending on the specific process [55].

As an alternative to heuristics, some researchers have shown that quantitatively estimating build metrics based on part geometry can also predict manufacturing process performance [56,57]. Since these calculations can be embedded in automated software [57], they allow designers to succeed in assessing their designs with fewer heuristics. Although many quantitative metrics are quick to calculate, like number of layers or part mass, others, like outputs of thermal finite element analysis (FEA) simulation of a build, require non-negligible computation [58]. Thus, applying an ML approach to estimate quantitative build metrics has the potential to offer a more efficient method of calculating metrics of various complexities for different AM processes. Moreover, the versatility of such an approach could help proliferate automated build performance calculations throughout industry, thereby reducing the need for relatively subjective heuristics.

The possibilities for using ML to benefit quantitative AM build metric assessment has attracted the attention of the academic community. For example, researchers have used multilayer perceptrons [6] to assess AM build time. McComb et al. [5] used variational autoencoders to estimate part mass, support material mass, and build time using a voxel-based input. Neither of these efforts used a 3D CNN to evaluate the continuous values. Multidimensional CNNs might be better suited for the high-dimensional inputs associated with 3D design data based on their successful use for regression in other fields, such as improved automatic cell counting with 2D CNNs [59]. Although not 3D, 2D inputs (e.g., images of cells) share similar high-dimensionality challenges to 3D data (e.g., voxel-based models) that can be remediated using a CNN [1].

Although the ML evaluation of AM build metrics could be applied to more complex metrics (e.g., strength assessed with FEA), the concept has not been demonstrated on simpler, more easily verified metrics. For this reason, we focus on part mass, required support material mass, and build time in this study. "Part mass" is the estimated mass of raw material required to 3D print the part at full-density, not including waste material. "Required support material mass" is the estimated mass of waste material required to support surfaces that are oriented less than 45 deg to the build platform. "Build time" is the estimated time required for the part to be additively manufactured using a generalized material extrusion process. These values have been previously analyzed in the literature for generalized AM processes and are relatively easy to estimate for the large numbers of designs needed to train 3D CNNs. Further details defining the context of these build metrics for this study are documented in Sec. 2.2.

In addition to investigating the nature of repositories used to train neural networks, we apply 3D CNNs to AM-focused geometries for quantitative analysis. To build on prior 3D ML research, we prioritize the use of larger sample sizes that are more akin to realistic AM design repositories and investigate how attributes of the repositories themselves are related to their usefulness as 3D CNN training datasets.

## 2 Methodology

This section describes the techniques and rationales used to develop "artificial" design repository (ADR) datasets (Sec. 2.1); the part mass, support material mass, and build time calculations used to label CNN training inputs (Sec. 2.2); and the 3D CNN architecture (Sec. 2.3).

**2.1 Artificial Design Repositories.** Although build-assessment ML will be most effective when deployed on empirical design repositories, we created our own ADR to achieve greater sample size and consistency. We define an ADR to be a design repository containing designs that were not individually created or holistically compiled with the intent of future manufacture. We consider the ShapeNet and ModelNet repositories to be examples of ADRs because their primary purpose is to support geometry-driven, classification benchmarking [25,26]. The current study aims to uncover generalizable trends about design repository attributes. The careful control over design repositories' attributes afforded by an ADR makes it more desirable than less-controllable, empirical datasets for this goal.

*2.1.1 Parametric Design Scheme.* The first step in creating our ADR was defining what types of designs would be included. We chose a set of shapes to capture several features of interest to AM. First, we wanted to include both planar and curved surfaces because each can have different influences on final part surface quality in AM processes [60]. Second, we decided to explore designs of varying bounding box aspect ratios, an attribute that has been found to have substantial influence on build time, often due to slower $z$ direction machine motion compared to motion parallel to the build platform [61]. Third, we included both concave and convex geometries to capture both part-surface-to-build-platform and part-surface-to-part-surface support material scenarios (see Fig. 5).

To achieve these repository goals, we designed parts based on 18 parametric design templates (see Fig. 1). Modeling the geometries parametrically allowed us to procedurally generate thousands of

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

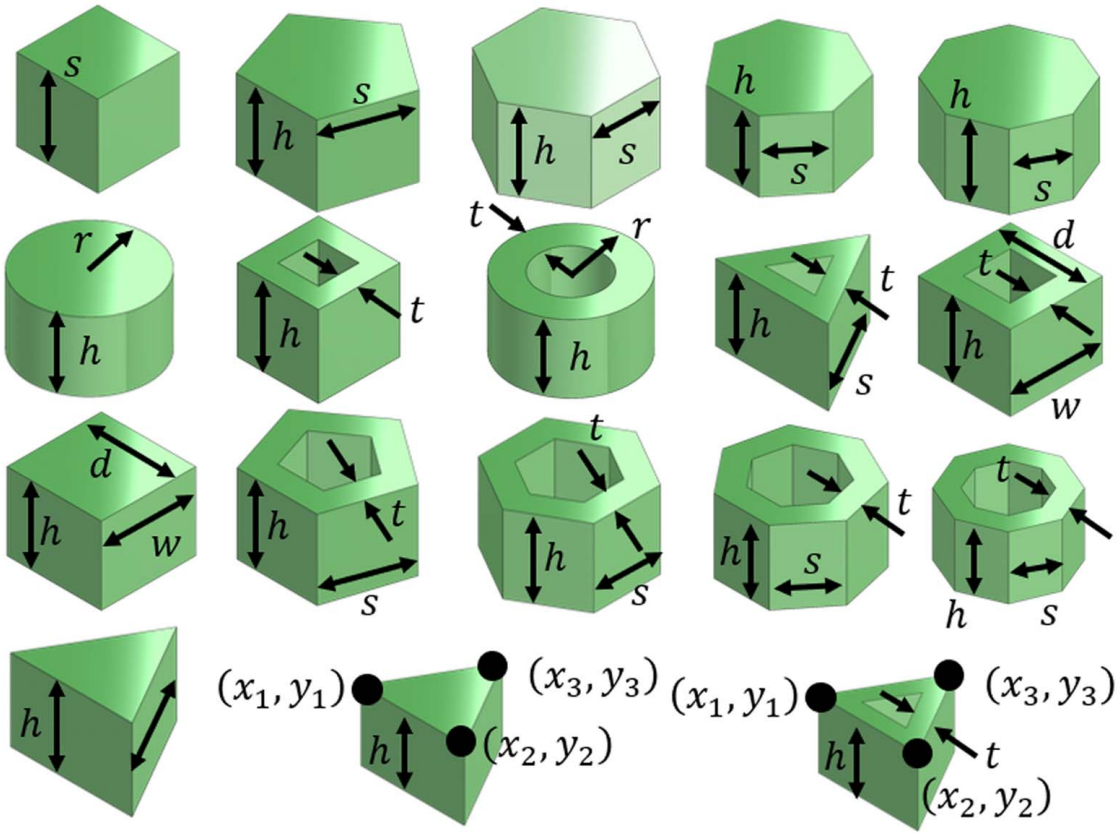



**Fig. 1 Example CAD models of the 18 parametric design templates. Adjustable parameters side length (*s*), height (*h*), radius (*r*), width (*w*), depth (*d*), wall thickness (*w*), and vertex coordinates ($x_i$, $y_i$) are labeled.**

designs that were unique, could be assessed for build metrics, and would be ensured to fit our design criteria (see Fig. 2). The design templates had variable but finite numbers of continuous parameters. They ranged in complexity from the simplest, a cube defined only by side length $s$, to the most complex, an irregular triangular prismatic tube, which was defined by three vertex positions ($x_i$, $y_i$), height $h$, and wall thickness $t$.

Although these designs pale in comparison to the complexity possible through AM, we assert that our methods are appropriate for an initial study because they show how diversity of complexity may be applied to an ADR in a controlled, incrementally increasing fashion. Parametric design spaces such as the templates used here are prevalent in engineering design. For example, I-beams [62], pipes [63], and tensile test specimens [64] are all common

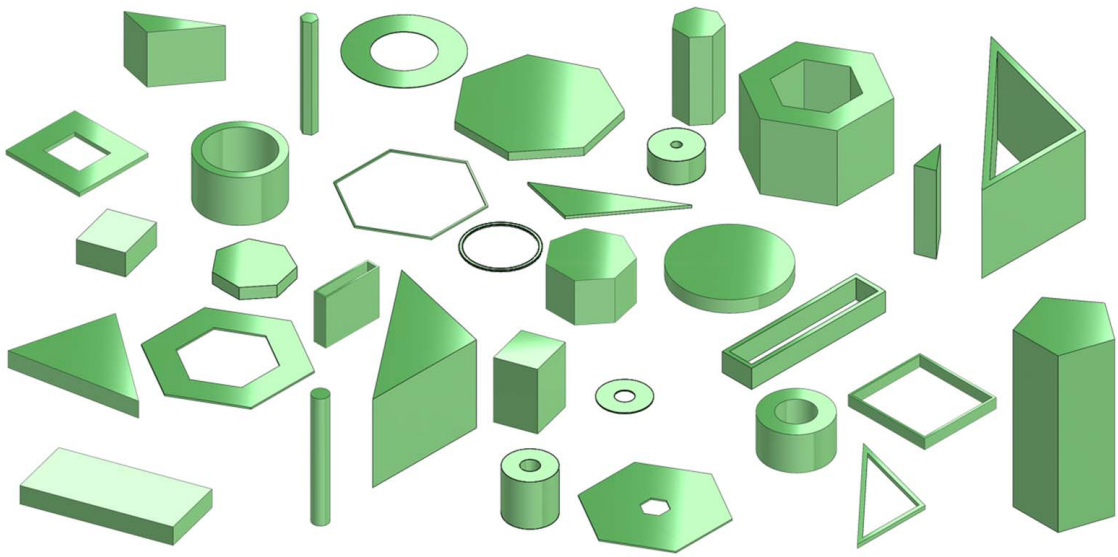


**Fig. 2 Example CAD models demonstrating a subset of the many possible geometry variants achieved by randomizing the design templates' adjustable parameters**



Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

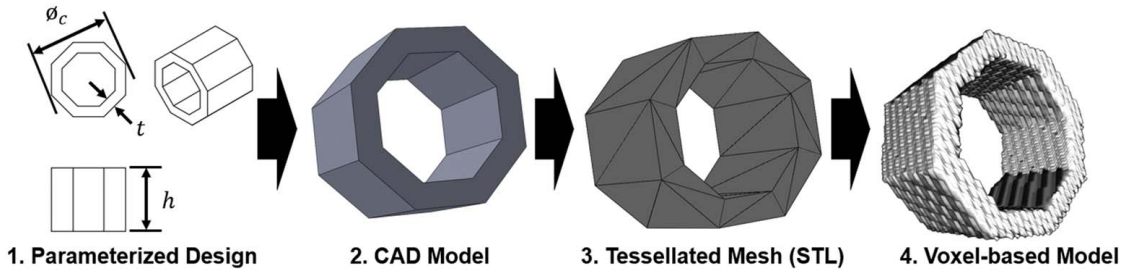



**Fig. 3 Illustration of the digital thread used to transform a parameterized design (1) to a voxel-based model (4)**

engineering design products that have been the subject of standardization. The applicable standards reduce the infinite spectrum of possible geometries meeting those descriptions to a clearly defined structure of parameters. The templates used here are similar in several respects to templates used in real-world applications. First, they are controlled by a similar number of parameters as ubiquitous, real-world engineering components. For instance, the geometry of a pipe is routinely represented by a circular outer diameter, a uniform wall thickness, and a length. These parameters are identical to our "cylinder" design template (see Fig. 1). Second, many complex geometries can be represented as additive and subtractive combinations of 3D primitive shapes, as is done when using the constructive solid geometry (CSG) modeling methodology [65]. Thus, work done on design templates has the potential to be expanded to more complex geometries through application of CSG or similar approaches.

Designs were modeled in four stages to instantiate the ADR (see Fig. 3). First, design parameters were computed and stored. Next, the parameter values were used to create a watertight tessellated mesh conforming to the appropriate design template attributes. The geometries were initially defined centered at the origin before rotation and translation were applied, if prescribed, in that order. The STL file was used for this step. Finally, the STL files were converted to a Boolean voxel representation using a custom algorithm. The voxel models were each a $64 \times 64 \times 64$ element matrix in which each element represented the presence or absence of material at the center of the voxel. Material presence was represented by 1, and material absence was represented by 0. The voxel space was $10\ \text{cm} \times 10\ \text{cm} \times 10\ \text{cm}$. This voxel space was chosen to balance geometric fidelity with the size of a realistic, desktop material extrusion AM machine.

*2.1.2 Design Control Treatments.* Since the same geometry can be additively manufactured in a variety of orientations that are not always equally advantageous, the final, often critical steps in DfAM are orienting and placing parts on the build platform [66]. Thus, we consider orientation and position to be design parameters in their own right.

For this study, we decided to vary the amount of control enforced over build orientation and position as factors in an experiment. Varying whether orientation is controlled simulates the real-world phenomenon of novice designers building with suboptimal orientation compared to expert designers [66]. Fabricating AM parts with suboptimal orientation can result in increased support material, poor surface finish, and undesirable anisotropy [67]. Position is varied for a different reason. Since CNNs use a fixed geometric space [4], we chose to investigate consistency of position within that space. The variability across designs within a repository was introduced by procedural generation using a pseudorandom number generator to select parameter values (position coordinates, orientation rotations, and design template parameter lengths). Although the resultant variability was not specifically characterized for this study, the number generator was designed to provide a uniform distribution of values and was selected to minimize the likelihood that particular portions of the geometric design space would be explored more than others.

We prescribed four different treatments to the designs in the ADR to restrict or not restrict position and orientation design control (see Fig. 4) to create four different types of ADRs:

- Treatment N ("No transformation") ADRs consisted of all designs modeled with their centers of mass located at the origin and oriented such that no support material is required. Qualitatively, treatment N was considered analogous to expert-level design control with the objective of minimizing the support material.
- Treatment T ("Translation") ADRs translated designs randomly about the design space, orienting them similarly to designs in treatment N. This treatment simulates designers who abide by good DfAM practices but are unaware that a design's position could impact a CNN.
- Treatment R ("Rotation") ADRs included designs that were positioned similarly to treatment N and randomly rotated about the $x$, $y$, and $z$ axes. These designs simulate poor design repository control practices, in which the designer does not care about orientation.
- Treatment T + R ("Translation Plus Rotation") ADR designs were randomly translated and rotated, representing the worst-case, in which both position and orientation are not controlled in the repository curation process.

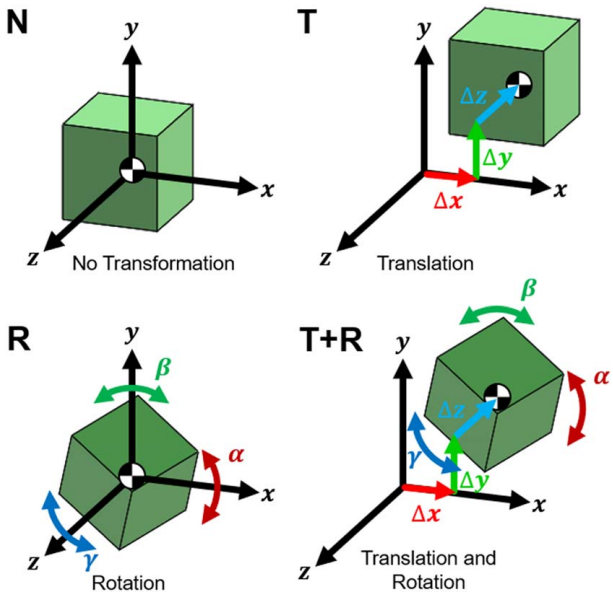



**Fig. 4 Illustration of the four ADR treatments N (top-left), T (top-right), R (bottom-left), and T + R (bottom-right). The possible transformations $x$ direction translation ($\Delta x$), $y$ direction translation ($\Delta y$), $z$ direction translation ($\Delta z$), rotation about the $x$ axis ($\alpha$), rotation about the $y$ axis ($\beta$), and rotation about the $z$ axis ($\gamma$) are labeled.**

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

Four replicate ADRs were created for each treatment type, for a total of 16 ADRs.

Each individual ADR contained 250 variants of each design template, for a total of 4,500 unique designs per repository. All parameter values and, if applicable, rotation or translation transformations were randomly generated so that all designs (a) had at least some nonzero mass when converted to the voxel format described in Sec. 2.1.1 and (b) had parameter values that were not so large as to place any portion of the design outside of the design space bounding box described in Sec. 2.1.1. The total number of unique geometries used in the study was 72,000. This number is near or larger than similar works in the field of geometric machine learning, which often use between hundreds and tens of thousands of unique geometries [4–7,10,15]. This number of unique geometries is also comparable to the number of geometries in realistic design repositories in industry [2].

Each ADR was then used to train three CNNs (one to predict each build metric). These networks, described in Sec. 2.3, were each used to evaluate three other ADRs of each of the four ADR types. These evaluation ADRs, which themselves were used to train CNNs, were ensured to be totally unseen during training of the CNNs used to evaluate them. In other words, all ADRs were used as both training and testing datasets, but they were never mixed during the training process. In total, 576 repository evaluation runs were completed on the CNNs, for a total of 2,592,000 individual point estimations made by our CNN architecture. The evaluation results were used to calculate a coefficient of determination (COD) for each CNN-evaluation repository pair. These three repeated COD values for each predicted ADR treatment were averaged to produce a single COD value representing each network's capability to predict data from a particular treatment type. For each network, the four single-treatment average COD values and the overall average COD represent five distinct experimental outputs that were examined. Through this experimental method, we are able to assess both the within-treatment prediction capabilities and across-treatment prediction capabilities of the CNN architecture.

When analyzing the mean evaluation results for general trends, we considered whether the two factors of rotation and translation and their interaction were statistically significant using a two-way analysis of variance (ANOVA). We also investigated whether mean CODs were statistically the same or different by using Tukey's honest significant difference (HSD) tests [68]. All statistical significance tests used a significance level of 0.05 and were accompanied by a check for independence, within-treatment normal distribution of the output, equal variance, and outliers. These statistical comparisons of COD values were then used to draw conclusions regarding how modifying the dataset impacted the ability of a trained neural network to make predictions for data that was both like and unlike the respective data.

**2.2 Additive Manufacturing Build Metrics Calculation.** For the purposes of this initial study, we assumed the AM process of interest to be material extrusion with a constant material deposition rate, one hundred percent part infill, support material capabilities enabled for overhanging features, and a $z$ axis movement step between all layers.

We selected three AM build metrics with different degrees of orientation dependence to model using the CNN architecture. These metrics were calculated for individual designs using specialized algorithms before being used as training labels for the CNNs as discussed in Secs. 2.2.1–2.2.3. For consistency and maximum precision, all build metric calculations used an STL file, a de facto standard in DfAM [69]. This occurred prior to voxelization.

*2.2.1 Part Mass.* Part mass is the simplest build metric included in the study, chosen because it is independent of part orientation. A custom implementation of the STL volume calculation methodology by Kechagias and Chryssolouris [70] was used. This algorithm was selected because it is not proprietary and can readily be reused for arbitrarily complex geometries in future studies. For this calculation, we assumed that the material was polylactic acid (PLA), with a constant part density of 1.25 g per cubic centimeter with 100% infill.

*2.2.2 Required Support Material Mass.* In material extrusion, overhanging surfaces angled below some self-supporting angle with respect to the build platform require extra support material. Support material is often a 3D polymeric, break-away scaffold structure built concurrently with the main structure to prevent features from collapsing under their own weight [71]. For our material extrusion process, we assumed the minimum self-supporting angle to be 45 deg. Similar to part mass, we also assumed the support material was PLA but built in a lattice structure with 30% infill by volume. A numerical, discretized ray-plane intersection approach was used to determine overhangs and support volume (see examples in Fig. 5). In this algorithm, intersections between evenly spaced test rays and overhanging STL triangles that were less than the minimum self-supporting angle were used to model the volume under overhanging surfaces. The calculation is summarized by

$$V_s = \frac{l_x l_y}{\delta_x \delta_y} \sum_{i=1}^{n_c} d_i \tag{1}$$

where $V_s$ is the estimated support material volume, $n_c$ is the number of ray intersections between an overhanging triangle that was less than the minimum self-supporting angle and a test ray, $l_x$ is the length of the model bounding box parallel to the $x$ axis, $l_y$ is the length of the model bounding box parallel to the $y$ axis, $\delta_x$ is the step distance between test rays in the $x$ axis direction, $\delta_y$ is the step distance between test rays in the $y$ axis direction, and $d_i$ is the lesser distance in the negative $z$ direction between the ray-triangle intersection point $i$ and either another surface of the model or the build platform.

For this calculation, the orientation of the part was assumed to be as-designed in the STL file, and the build platform was parallel to the $xy$ plane and coincident with the STL file's minimum vertex $z$ coordinate. Additionally, the positive $z$ axis direction was considered up, i.e., the direction in which successive layers are added. A total of 250,000 evenly spaced rays were intersected with the bounding box region of each STL model. This algorithm was chosen for its simplicity, usability with the STL file format, lack of reliance on proprietary, machine-dependent algorithms, and ability to interchange a different self-supporting angle if needed for future studies.

*2.2.3 Build Time.* Similar to support material volume, numerous build time estimation methods have been proposed [61,72–74]. We selected and modified a formula that was intended for generalized material extrusion AM predictions. Alexander et al. [61] detail build time for general fused deposition modeling AM

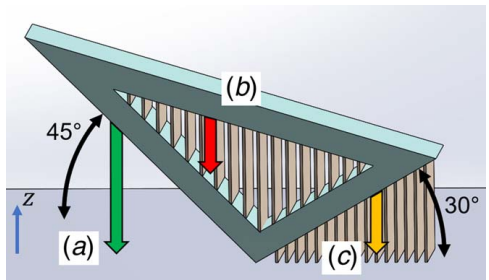



**Fig. 5 Illustration of a design that requires support material showing (*a*) an overhanging triangle greater than the minimum self-supporting angle, (*b*) an overhanging triangle that requires part-surface-to-part-surface support, and (*c*) an overhanging triangle that requires part-surface-to-build platform support**

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

processes; however, our process differed in several ways. First, we did not consider a process with a wipe function (an accurate assumption for desktop material extrusion AM) so that all wipe-oriented time contribution went to zero. Second, our support material was printed as a 30% infill lattice so that a given volume requiring support could be printed faster than a given volume of primary part material. Third, we assumed a constant material deposition rate by volume. These assumptions collectively allow us to modify the equation to

$$T_{\text{build}} = t_m V_p + t_s V_s + N_{\text{lay}} t_{z\ \text{move}} \tag{2}$$

where $T_{\text{build}}$ is the time to build a part, $t_m$ is the time to deposit 1 $\text{cm}^3$ part volume, $t_s$ is the time to deposit 1 $\text{cm}^3$ support material lattice macro-structure volume, $V_p$ is the part volume, and $V_s$ is the support material lattice macro-structure volume.

The coefficients in Eq. (2) are process-specific. For our study, we conducted an ad hoc exploratory exercise to determine realistic coefficients using MakerBot Desktop Version 3.10.1 to estimate build times for prismatic designs of various sizes and orientations. Based on this range-finding activity, we set our case-study build time coefficients $t_m$, $t_s$, and $t_z$ equal to 0.065 $\text{h/cm}^3$, 0.02 $\text{h/cm}^3$, and 0.002 h per layer, respectively.

We chose build time as a metric for the study because it represents a middle ground between the other two metrics in terms of orientation dependence. It is affected by both an orientation-independent property, i.e., the part mass, and two orientation-dependent properties, namely, the support material mass and the $z$ height.

**2.3 Convolutional Neural Network Architecture Design and Training.** We implemented a 3D CNN architecture to map an input of voxel-based 3D geometry to a single continuous estimated build metric output (see Fig. 6 for architecture description). The same CNN base architecture was used to train all 3D CNNs in the study. For our model, we prioritized input resolution over network depth to enable the network to learn patterns for a larger range of input design scale and thickness.

We used Keras version 2.2.4 [75] with TensorFlow version 1.12 [76] and Python version 3.6.5 for all CNN training and testing. Our CNN architecture consisted of 10 layers. This architecture accepts a voxel-based 3D geometric input, which contains 262,144 individual floating-point values, corresponding to the volumetric occupancy of a single design (see Sec. 2.1). Since the current work is not focused on optimizing the CNN design itself, we chose to simplify the network design wherever possible. To this end, we chose to use consistent $3 \times 3 \times 3$ window sizes for all convolutional layers. Training was facilitated with the "Adam" optimizer [77]. For each training set, we used a training ratio of 0.75 and a total training sample size of 3375. Validation sample size was 1125. Multiple evaluation sets were used for each trained network as part of our experiments described in Sec. 2.1.2. We ensured that no validation set included any individual designs from that network's training set. For consistency, all CNNs were trained for 10 epochs, which exceeded the convergence point found in baseline architecture testing. CNN training and validation were completed on an NVIDIA Quadro P6000 graphics card.

The CNN model consisted of a 3D convolutional section followed by a fully connected section. The convolutional section had three convolution-pooling layer pairs. This section splits the model into 64 reduced dimensionality channels, within which the network can learn unique feature patterns. Next, the fully connected section connected a flattened input layer, a hidden layer, and a single neuron output layer to produce a predicted build metric quantity. Hidden layers used rectified linear unit activation, and the output layer used a linear activation [1].

**2.4 Baseline Linear Regression Model.** In addition to testing the CNN architecture's predictive capabilities for tens of thousands of samples and millions of point estimates, we explore how well the CNN compares to a simpler estimation technique that does not rely on neural networks. Specifically, a multiple linear regression model is used as a baseline against which the CNN results are compared.

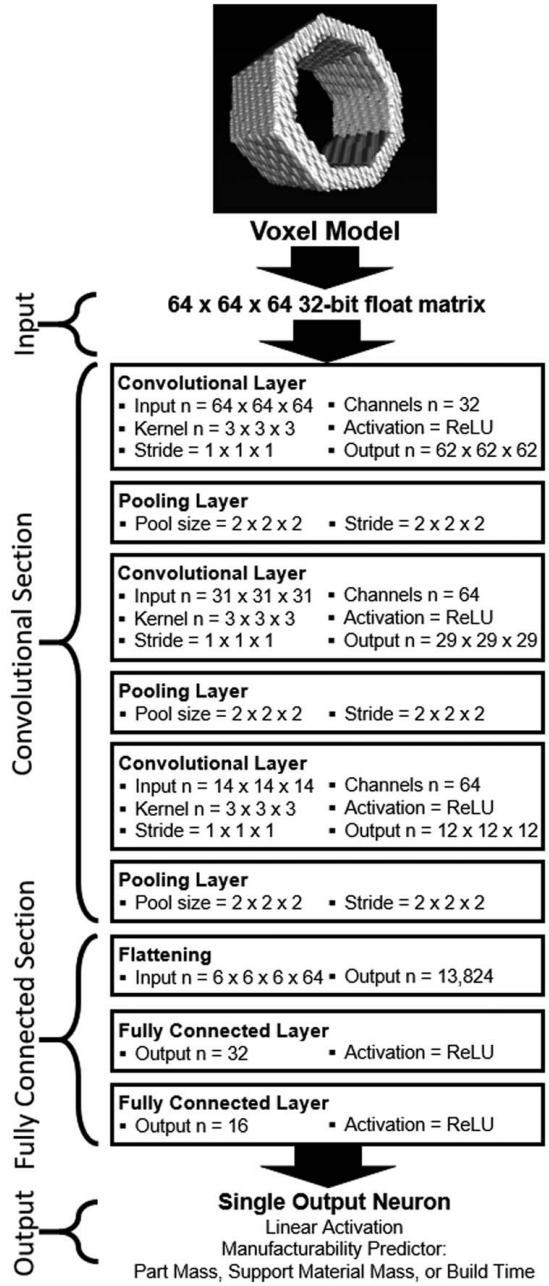



**Fig. 6 Quantitative build metric estimation CNN architecture**

We selected a multiple linear regression model because it is a widely used technique that may be applied to spaces with arbitrarily high dimensionality [78]. Additionally, the linear regression model was specifically chosen because (1) it can readily accept a Boolean input identical to that used by the CNN and (2) it uses a similar number of trainable parameters as the CNN, to within an order of magnitude. For reference, the CNN used 262,144 individual floating-point values in its input, represented as $64 \times 64 \times 64$ element, 3D matrix. In the linear regression, this input was flattened to a one-dimensional, 262,144 element vector. The CNN incorporated 609,857 trainable parameters and the linear regression used 262,145 trainable parameters.

This comparative baseline model accomplishes two goals. First, it compares the CNN to a simpler regression analysis technique that is not designed specifically to learn spatial features. If the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

linear regression were capable of predicting the metrics nearly as well as the CNN, then one might conclude that the CNN's spatial pattern learning capabilities do not add value to the fitting process. Second, the linear model provides context for the degree to which our chosen build metrics may be predicted without a neural network. Again, if a linear model is sufficient to predict these metrics, then one might conclude that the additional complexity of a CNN is unwarranted.

The multiple linear regression model was fitted using a least squares approach. Prior to use as training data samples, geometric designs were flattened from the 3D voxel space described in Sec. 2.1.1 to 1D vectors with one representing material presence and zero representing material absence. The SCIKIT-LEARN v0.20.2 PYTHON package [79] was used to train linear models and evaluate prediction values. Other than the aforementioned conversion to 1D vectors, the training and evaluation datasets were identical to those used for CNN training and evaluation.

## 3 Results and Discussion

Summaries of the results are presented individually by build metric (part mass, support material mass, and build time). The multiple linear regression analysis results are presented first to establish a baseline, followed by CNN results. Although not all statistical results for this preliminary work are able to be shown, specifically discussed trends are supported by the statistical tests described in Sec. 2.1.2. The tabular figures (see Figs. 7–12) are to be interpreted as COD values for all possible training and evaluation ADR treatment combinations. A COD greater than 0 means that the network is better at estimating a dataset than a mean model (simply assuming new values are equal to the mean of the training values), and a COD equal to 1 is a perfect prediction. Note that the CODs in the figures are cumulative means of all repeats and

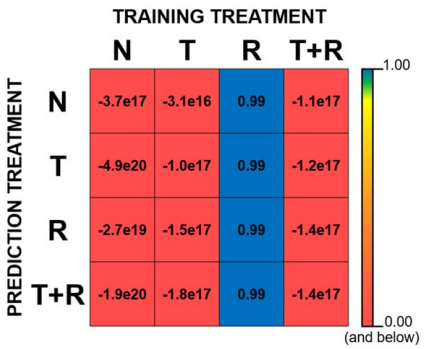



**Fig. 7 Average COD values for each part mass prediction baseline linear regression model treatment type when predicting part mass from all treatments**

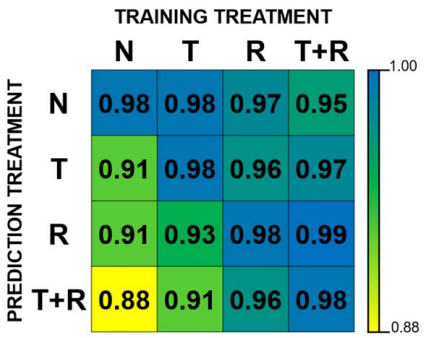



**Fig. 8 Average COD values for each part mass prediction CNN treatment type when predicting part mass from all treatments**

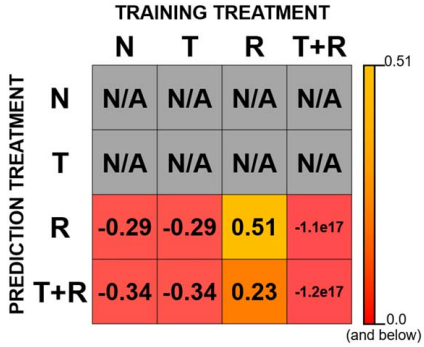



**Fig. 9 Average COD values for each support material mass prediction baseline linear regression model treatment type when predicting support material mass from all treatments**

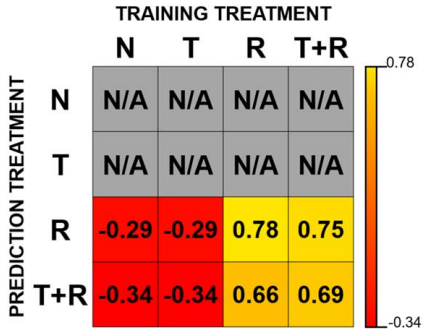



**Fig. 10 Average COD values for each support material mass prediction CNN treatment type when predicting support material mass from all treatments**

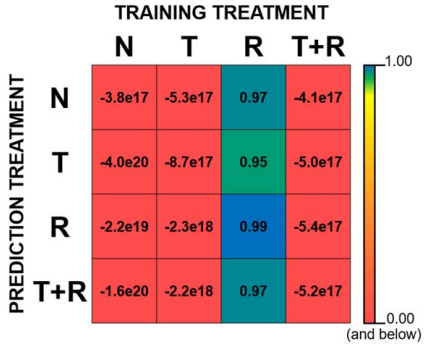



**Fig. 11 Average COD values for the build time prediction baseline linear model when predicting build time from all treatments**

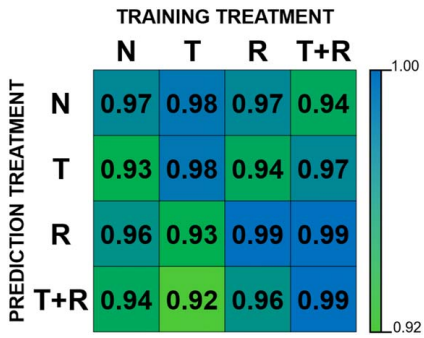



**Fig. 12 Average COD values for the build time prediction CNN treatment type when predicting build time from all treatments**



Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

replicates for that factor combination (12 COD values each), while the CODs used to calculate ANOVA and Tukey's HSD results are means of only the repeats for a given trained model (3 COD values each).

**3.1 Part Mass Results.** Multiple linear regression analysis revealed two dominant trends. In the first trend, models trained with treatments N, T, and T + R ADRs were incapable of predicting part mass for repositories that were not used during training (see Fig. 7).

Second, models trained on treatment R ADRs exhibited mean COD values of 0.99 when predicting part mass for all types of repositories. These results indicate that rotation of the training dataset without translation was the only repository preparation practice that resulted in useable linear models. Qualitatively, the linear regression model is not particularly robust and only produces useful predictions for 25% of the training and evaluation combinations. The linear regression models were all effective at estimating values from their own training data (all mean CODs between 0.97 and 0.99), which indicates that the treatment N, treatment T, and treatment T + R ADRs were likely very overfit. We suspect that the fact that the number of linear model independent variables (262,144) is two orders of magnitude larger than the number of training set samples (4500) makes this overfitting likely.

On average, the part mass CNNs were more accurate when training repositories were constructed with more design freedom (see Fig. 8). Treatments N, T, R, and T + R exhibited overall average CODs of 0.92, 0.94, 0.97, and 0.97, respectively. For overall average prediction COD, the only significant factor ($F = 9.99$, $p$-value = 0.00819) was the presence of training dataset rotation. This summary assessment suggests that rotation's effect on CNN performance was greater than translation. We believe that the orientation-independent nature of part mass contributes to this observation. Since all orientations should ideally result in the same part mass prediction value, a network receives more overall benefit if shown arbitrarily orientated parts during training, as opposed to arbitrarily translated parts.

In general, CNNs trained on one treatment were most effective (or highly effective) at making predictions on the same treatment. This is evidenced by the high and statistically equivalent values along the diagonal in Fig. 8. The CNNs trained on treatment N, the least general design repository, were very effective at predicting designs from their own treatment, but significantly (HSD $p$-value [N:R] = 0.0283, HSD $p$-value [N:T + R] = 0.0203) less effective at predicting those from foreign (different from their own) treatments when compared to rotation treatments foreign prediction capability. In contrast, treatment T + R was relatively effective at predicting all treatments. Interestingly, treatment T + R was least effective at predicting treatment N, although this difference's $p$-values were not significant.

Collectively, these part mass results suggest a potential trade-off in preparing the best part mass training dataset from an orientation and translation standardization perspective. For most cases in which the design control of future geometries will either be low or unknown, preparing the training dataset with minimal spatial design restrictions, as was done in treatment T + R, will likely result in the most accurate CNN. However, if the future designs will be guaranteed to be consistently oriented and positioned, training with a more standardized dataset, such as treatment N, will probably result in the best performance.

Additionally, comparison to the baseline linear regression indicates that the CNN is more robust than the linear regression model. None of the training and evaluation combinations resulted in a trained CNN that was less accurate than a mean model, whereas three quarters of the baseline model combinations were less accurate than a mean model. This study does not specifically investigate the reason that the linear models trained on treatment R were relatively accurate. However, we hypothesize that the success of the linear model is due to a combination of the normal distribution of the training and evaluation dataset dependent variable values and a fortuitous effect of the rotation transformation. Further study is necessary to assign specific cause.

**3.2 Support Material Mass Results.** As opposed to part mass, which resulted in relatively effective CNNs for all treatments, support material mass presented a more substantial challenge for prediction. None of the linear regression models were able to predict support material mass for treatment N and treatment T repositories. The combinations in Fig. 9 reflect the fact that geometries in treatments N and T both require no support material at all. Therefore, unless the models were capable of predicting support material perfectly, a COD value of negative infinity was unavoidable. The only linear regression models capable of predicting support material better than a mean model were those trained on treatment R repositories when used to predict treatment R and T + R repositories. These models exhibited COD values of 0.51 and 0.23, respectively.

As shown in Fig. 10, CNNs trained with treatments N and T were unable to predict repository values for treatments R and T + R more effectively than a mean model. The poor performance of networks trained on treatment N and T ADRs is because of the lack of variability in the training set, which essentially did not introduce any information regarding support material. CNNs trained with treatment R and treatment T + R ADRs were able to predict support material mass better than a mean model, but at greatly lower accuracies than was observed for part mass or build time. Since so few training and evaluation repository combinations resulted in either valid or positive COD results, statistical significance is not considered useful for informing support material conclusions.

Although CNN support material mass predictions were clearly worse than part mass for all cases, some similarities to the part mass trends are also apparent. For instance, the patterns for networks trained on treatments R and T + R resemble those for part mass (see Fig. 8). Specifically, these CNNs were all good at predicting datasets similarly treated to the training dataset, and the presence of rotation had a significantly greater influence on improving the network performance.

Compared to the linear regression model, the CNN exhibited similar overall trends except for two main differences. First, when treatment R repositories were used as training datasets to predict treatments R and T + R, the CNN outperformed the linear baseline model. Second, CNN models trained on treatment T + R repositories were able to predict support material mass better than a mean model but similarly trained linear regression models were not.

Taken together, we surmise that these trends, supported by both the CNN and baseline linear regression models, indicate that the interconnection between orientation and a design's likelihood of having a zero value for support material mass makes support material mass a much harder metric to predict than part mass, which is an orientation-independent metric that is always greater than zero. Furthermore, we speculate that support material is driven by a smaller subset of geometric attributes than part mass.

These conjectures suggest three hypotheses for future study. First, the CNN is anticipated to be most successful if trained on a mix of high support material and low support material parts while also ensuring inclusion of both zero and nonzero training values. Second, the CNN architecture used in this study, although relatively capable for part mass, is not complex or optimized enough to predict support material mass to an extent that would be useful for real-world applications even if given the best possible training dataset. Thus, further CNN studies, possibly using deeper neural networks, may be more advantageous. Finally, the ability of some CNNs to predict support material mass better than a mean model suggests that support material mass prediction, although challenging, may yet be a practicable feat for other voxel-based 3D CNNs.

**3.3 Build Time Results.** Although its calculated value incorporated influence from both part mass and support material mass, build time prediction CNNs behaved more similarly to part mass

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

(see Fig. 12). The baseline linear regression model exhibited nearly identical trends to those observed for build time (see Fig. 11). Linear models trained on treatment N, T, and T + R repositories were all unable to predict build time better than a mean model. On average, all linear models trained on treatment R repositories were able to predict build time better than a mean model when predicting repositories of all treatment types. One difference in baseline performance between part mass and build time was that the build time mean COD values for models trained on treatment R were slightly lower.

Build time CNNs for treatments N, T, R, and T + R averaged overall CODs of 0.95, 0.95, 0.96, and 0.97, respectively. Like part mass CNNs, build time CNNs were all better performing than a mean model and strongest at predicting values for identically treated repositories. Similar trends to part mass were also observed when CNNs were trained with designs in treatments T, R, and T + R ADRs. These included the observations that treatment T + R repositories resulted in the best predictive capabilities for all cases except when evaluating treatment N ADRs. We also note that the presence of rotation in the training dataset resulted in greater overall performance.

Despite this success, build time patterns varied from past mass results in decidedly different ways as well. One major difference is that CNNs trained with treatment N ADRs were better at predicting build time of treatment R and treatment T + R ADRs than part mass. Additionally, training set translation and the interaction between translation and rotation were more influential, appearing as statistically significant factors for more evaluation treatment outputs than part mass. This second observation means that it is not as clear that rotation has a greater overall influence than translation.

We hypothesize that these differences are due to the particular weighting ratio of part volume and support material volume influence on the calculated build time values that were obtained for the study. Since support material volume, which is orientation-dependent, contributes approximately one third as much to the overall build time as primary part volume, which is orientation-independent, the benefits of rotating the training set models may be less exaggerated. This lower exaggeration is manifested in the relatively poor performance of networks trained on treatments R and T + R in predicting treatments T and N, respectively. Similarly, the greater influence of part volume on the calculated build time value could be boosting the overall performance of treatment N. Replicating these results and identifying specific causation is part of ongoing work.

## 4 Conclusions and Future Work

This study demonstrates an approach to test how attributes of "artificial" design repositories make them more or less advantageous for training an ML construct. As an investigation of this topic, we used 3D CNNs to predict three different AM build metrics for ADR designs. First, a parametric design scheme was defined, encapsulating different surface types and geometry complexities within 18 design templates. Next, 72,000 unique designs were generated by systematically varying those parameters. Those designs were then organized into 16 uniformly sized ADRs, to which one of the four treatments was randomly applied. The treatments included all combinations of the presence or absence of random rotation and random translation of individual geometries. These treatments represented simplified analogs to design repositories crafted with different degrees of design process control. The ADRs were then used to train 3D CNNs to predict three AM build metrics: (1) part mass, (2) support material mass, and (3) build time. Next, the trained CNNs were used to evaluate the metrics for design repositories of all treatment types that were not used for training. Finally, the results were examined both statistically and qualitatively to identify general trends and potential areas for future study.

Regarding our first research question, overall network accuracy increased when less standardization was incorporated in the design process. Furthermore, random rotation of the training data tended to have greater overall influence in creating more effective CNNs than translation. Although this trend was observed in most of the combinations attempted, we did note that in the particular case in which evaluation data were most standardized, a more standardized training dataset resulted in better average network prediction performance.

In terms of our second research question, we observed that orientation-independent metrics were more accurately predicted by our CNN architecture than orientation-dependent metrics. Part mass, which was completely orientation-independent, and build time, which has a relatively high weighting of orientation-independent terms, were decisively easier to predict than support material mass, an orientation-dependent metric.

Finally, concerning our third research question, we conclude that the CNN was more consistently accurate at predicting all three build metrics than the baseline linear regression model for the training and evaluation conditions explored. The CNN was able to predict part mass and build time better than a mean model for all repositories and was able to predict support material mass better than a mean model for one quarter of repositories. The baseline linear regression model was only able to outperform a mean model for one quarter of the part mass and build time conditions and one eighth of the support material mass conditions. This comparison shows that the CNN was generally more accurate than a simpler model, and therefore, the spatially focused nature of 3D CNN analysis is relatively well suited for voxel-based regression. The observation that the linear regression was able to closely match the part mass and build time prediction performance of the CNN in some cases suggests that the CNN used here may be unnecessarily complex, and that the ideal prediction algorithm in terms of complexity may lie somewhere between a 3D CNN and linear regression. Further studies directly comparing a broader variety of machine learning algorithms could identify where the optimum lies for different build metrics.

This initial study presents an experimental methodology to define, prepare, and analyze design repositories for the purpose of identifying trends regarding the use of such repositories to train an ML construct. Further work is needed to confirm whether these results are generalizable to real-world design repositories and to assign causality to the trends observed. The current work's findings are limited in several ways. First, the work only investigates "artificial" design repositories with procedurally generated designs of relatively low complexity. Applying this methodology to real world, more complex and diverse designs could better identify the utility of the current approach. Specifically, follow-up studies investigating more diverse designs from actual design competitions or classroom settings could answer this question. These studies could incorporate designs that are of greater geometric complexity, especially those which do not exhibit clear optimal orientation upon human inspection. Second, this work investigated three AM build metrics. This effort could be expanded to other, possibly application-specific and functionally driven metrics, such as part strength or fatigue performance. Future work incorporating experimental studies of measured build metrics from physically fabricated, high production volume parts could validate the applicability of the theoretical findings. Finally, our study considers a single 3D CNN architecture. Future work could either optimize this CNN or interchange it with other ML constructs, such as attribute-based regression techniques that are capable of assessing the same or similar metrics more effectively.

## Acknowledgment

This material is based upon work supported by the National Science Foundation (Grant No. CMMI-1825535; Funder ID: 10.13039/501100001809). The authors acknowledge the generous support of the NVIDIA Corporation (Funder ID: 10.13039/100007065)

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

through the donation of the Quadro P6000 GPU used in this work. Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the authors and do not necessarily reflect the views of the sponsors.

## References


[1] Schmidhuber, J., 2015, "Deep Learning in Neural Networks: An Overview," Neural Networks, **61**, pp. 85–117.
[2] Regli, W. C., and Cicirello, V. A., 2000, "Managing Digital Libraries for Computer-Aided Design," Comput. Aided Des., **32**(2), pp. 119–132.
[3] Lyu, G., Chu, X., and Xue, D., 2017, "Product Modeling From Knowledge, Distributed Computing and Lifecycle Perspectives: A Literature Review," Comput. Ind., **84**, pp. 1–13.
[4] Dering, M. L., and Tucker, C. S., 2017, "A Convolutional Neural Network Model for Predicting a Product's Function, Given Its Form," ASME J. Mech. Des., **139**(11), p. 111408.
[5] McComb, C., Murphey, C., Meisel, N., and Simpson, T. W., 2018, "Predicting Part Mass, Required Support Material, and Build Time Via Autoencoded Voxel Patterns," 29th Annual International Solid Freeform Fabrication Symposium, Austin, TX, Aug. 13–15, pp. 1–15.
[6] Munguía, J., Ciurana, J., and Riba, C., 2009, "Neural-Network-Based Model for Build-Time Estimation in Selective Laser Sintering," Proc. Inst. Mech. Eng. Part B J. Eng. Manuf., **223**(8), pp. 995–1003.
[7] Tsai, H.-C., Hsiao, S.-W., and Hung, F.-K., 2006, "An Image Evaluation Approach for Parameter-Based Product Form and Color Design," Comput. Aided Des., **38**(2), pp. 157–171.
[8] Chan, S. L., Lu, Y., and Wang, Y., 2018, "Data-Driven Cost Estimation for Additive Manufacturing in Cybermanufacturing," J. Manuf. Syst., **46**, pp. 115–126.
[9] Samie Tootooni, M., Dsouza, A., Donovan, R., Rao, P. K., James Kong, Z., and Borgesen, P., 2017, "Classifying the Dimensional Variation in Additive Manufactured Parts From Laser-Scanned Three-Dimensional Point Cloud Data Using Machine Learning Approaches," J. Manuf. Sci. Eng., **139**(9), p. 091005.
[10] Maturana, D., and Scherer, S., 2015, "VoxNet: A 3D Convolutional Neural Network for Real-Time Object Recognition," 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hamburg, Germany, Sept. 28–Oct. 2, pp. 922–928.
[11] Jain, A. K., Mao, J., and Mohiuddin, K. M., 1996, "Artificial Neural Networks: A Tutorial," Computer, **29**(3), pp. 31–44.
[12] Pal, N. R., and Pal, S. K., 1993, "A Review on Image Segmentation Techniques," Pattern Recognit., **26**(9), pp. 1277–1294.
[13] Egmont-Petersen, M., de Ridder, D., and Handels, H., 2002, "Image Processing With Neural Networks—A Review," Pattern Recognit., **35**(10), pp. 2279–2301.
[14] McComb, C., 2019, "Toward the Rapid Design of Engineered Systems Through Deep Neural Networks," *Design Computing and Cognition'18*, J. S. Gero, ed., Springer International Publishing, Cham, pp. 3–20.
[15] Kleesiek, J., Urban, G., Hubert, A., Schwarz, D., Maier-Hein, K., Bendszus, M., and Biller, A., 2016, "Deep MRI Brain Extraction: A 3D Convolutional Neural Network for Skull Stripping," Neuroimage, **129**, pp. 460–469.
[16] Wang, T.-M., Xi, J.-T., and Jin, Y., 2007, "A Model Research for Prototype Warp Deformation in the FDM Process," Int. J. Adv. Manuf. Technol., **33**(11–12), pp. 1087–1096.
[17] Khosravi, A., Nahavandi, S., Creighton, D., and Atiya, A. F., 2011, "Comprehensive Review of Neural Network-Based Prediction Intervals and New Advances," IEEE Trans. Neural Networks, **22**(9), pp. 1341–1356.
[18] Austin, P. C., and Steyerberg, E. W., 2015, "The Number of Subjects Per Variable Required in Linear Regression Analyses," J. Clin. Epidemiol, **68**(6), pp. 627–636.
[19] Regli, W. C., and Gaines, D. M., 1997, "A Repository for Design, Process Planning and Assembly," Comput. Aided Des., **29**(12), pp. 895–905.
[20] Szykman, S., 2002, "Architecture and Implementation of a Design Repository System," Volume 1: 22nd Computers and Information in Engineering Conference, Montreal, Quebec, Canada, Sept. 29–Oct. 2, ASME, pp. 429–443.
[21] Bohm, M. R., Stone, R. B., and Szykman, S., 2005, "Enhancing Virtual Product Representations for Advanced Design Repository Systems," J. Comput. Inf. Sci. Eng., **5**(4), p. 360.
[22] Bohm, M. R., Stone, R. B., Simpson, T. W., and Steva, E. D., 2006, "Introduction of a Data Schema: The Inner Workings of a Design Repository," Volume 3: 26th Computers and Information in Engineering Conference, Philadelphia, PA, Sept. 10–13, ASME, pp. 631–642.
[23] Bohm, M. R., Vucovich, J. P., and Stone, R. B., 2008, "Using a Design Repository to Drive Concept Generation," J. Comput. Inf. Sci. Eng., **8**(1), p. 014502.
[24] Devendorf, M., Lewis, K., Simpson, T. W., Stone, R. B., and Regli, W. C., 2009, "Evaluating the Use of Digital Product Repositories to Enhance Product Dissection Activities in the Classroom," J. Comput. Inf. Sci. Eng., **9**(4), p. 041008.
[25] Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., and Xiao, J., 2015, "3D ShapeNets: A Deep Representation for Volumetric Shapes," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, June 7–12, pp. 1912–1920.
[26] Chang, A. X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., Xiao, J., Yi, L., and Yu, F., 2015, "ShapeNet: An Information-Rich 3D Model Repository."
[27] Aoyagi, K., Wang, H., Sudo, H., and Chiba, A., 2019, "Simple Method to Construct Process Maps for Additive Manufacturing Using a Support Vector Machine," Addit. Manuf., **27**, pp. 353–362.
[28] Menon, A., Póczos, B., Feinberg, A. W., and Washburn, N. R., 2019, "Optimization of Silicone 3D Printing With Hierarchical Machine Learning," 3D Print. Addit. Manuf. (submitted).
[29] Sharifi, S., and Banadaki, Y. M., 2019, *Smart Structures and NDE for Energy Systems and Industry 4.0*, C. Niezrecki, N. G. Meyendorf, K. Gath, eds., SPIE, Anaheim, CA, p. 33.
[30] Harrison, R., Holm, E. A., and De Graef, M., 2019, "On the Use of 2D Moment Invariants in the Classification of Additive Manufacturing Powder Feedstock," Mater. Charact., **149**, pp. 255–263.
[31] He, H., Yang, Y., and Pan, Y., 2019, "Machine Learning for Continuous Liquid Interface Production: Printing Speed Modelling," J. Manuf. Syst., **50**, pp. 236–246.
[32] Stavroulakis, P., Chen, S., Delorme, C., Bointon, P., Tzimiropoulos, G., and Leach, R., 2019, "Rapid Tracking of Extrinsic Projector Parameters in Fringe Projection Using Machine Learning," Opt. Lasers Eng., **114**, pp. 7–14.
[33] Baturynska, I., Semeniuta, O., and Martinsen, K., 2018, "Optimization of Process Parameters for Powder Bed Fusion Additive Manufacturing by Combination of Machine Learning and Finite Element Method: A Conceptual Framework," Procedia CIRP, **67**, pp. 227–232.
[34] Scime, L., and Beuth, J., 2019, "Using Machine Learning to Identify In-Situ Melt Pool Signatures Indicative of Flaw Formation in a Laser Powder Bed Fusion Additive Manufacturing Process," Addit. Manuf., **25**, pp. 151–165.
[35] Caggiano, A., Zhang, J., Alfieri, V., Caiazzo, F., Gao, R., and Teti, R., 2019, "Machine Learning-Based Image Processing for On-Line Defect Recognition in Additive Manufacturing," CIRP Ann., **68**, pp. 3–6.
[36] Lin, W., Shen, H., Fu, J., and Wu, S., 2019, "Online Quality Monitoring in Material Extrusion Additive Manufacturing Processes Based on Laser Scanning Technology," Precis. Eng.
[37] Zhang, B., Liu, S., and Shin, Y. C., 2019, "In-Process Monitoring of Porosity During Laser Additive Manufacturing Process," Addit. Manuf., **28**, pp. 497–505.
[38] Liu, C., Law, A. C. C., Roberson, D., and James Kong, Z., 2019, "Image Analysis-Based Closed Loop Quality Control for Additive Manufacturing With Fused Filament Fabrication," J. Manuf. Syst., **51**, pp. 75–86.
[39] Sturm, L. D., Albakri, M. I., Tarazaga, P. A., and Williams, C. B., 2019, "In Situ Monitoring of Material Jetting Additive Manufacturing Process Via Impedance Based Measurements," Addit. Manuf., **28**, pp. 456–463.
[40] Wu, H., Yu, Z., and Wang, Y., 2019, "Experimental Study of the Process Failure Diagnosis in Additive Manufacturing Based on Acoustic Emission," Measurement, **136**, pp. 445–453.
[41] Tapia, G., Elwany, A. H., and Sang, H., 2016, "Prediction of Porosity in Metal-Based Additive Manufacturing Using Spatial Gaussian Process Models," Addit. Manuf., **12**, pp. 282–290.
[42] Wu, M., Song, Z., and Moon, Y. B., 2019, "Detecting Cyber-Physical Attacks in Cybermanufacturing Systems With Machine Learning Methods," J. Intell. Manuf., **30**(3), pp. 1111–1123.
[43] Al Faruque, M. A., Chhetri, S. R., Canedo, A., and Wan, J., 2016, "Acoustic Side-Channel Attacks on Additive Manufacturing Systems," 2016 ACM/IEEE 7th International Conference on Cyber-Physical Systems (ICCPS), Vienna, Austria, Apr. 11–14, pp. 1–10.
[44] Al Faruque, M. A., Chhetri, S. R., Canedo, A., and Wan, J., 2016, "Forensics of Thermal Side-Channel in Additive Manufacturing Systems," CECS Tech. Report# 16-01.
[45] Francis, J., and Bian, L., 2019, "Deep Learning for Distortion Prediction in Laser-Based Additive Manufacturing Using Big Data," Manuf. Lett., **20**, pp. 10–14.
[46] Khanzadeh, M., Rao, P., Jafari-Marandi, R., Smith, B. K., Tschopp, M. A., and Bian, L., 2017, "Quantifying Geometric Accuracy With Unsupervised Machine Learning: Using Self-Organizing Map on Fused Filament Fabrication Additive Manufacturing Parts," J. Manuf. Sci. Eng., **140**(3), p. 031011.
[47] Zhu, Z., Anwer, N., Huang, Q., and Mathieu, L., 2018, "Machine Learning in Tolerancing for Additive Manufacturing," CIRP Ann., **67**(1), pp. 157–160.
[48] Hamel, C. M., Roach, D. J., Long, K. N., Demoly, F., Dunn, M. L., and Qi, H. J., 2019, "Machine-Learning Based Design of Active Composite Structures for 4D Printing," Smart Mater. Struct., **28**(6), p. 065005.
[49] Li, Z., Zhang, Z., Shi, J., and Wu, D., 2019, "Prediction of Surface Roughness in Extrusion-Based Additive Manufacturing With Machine Learning," Robot. Comput. Integr. Manuf., **57**, pp. 488–495.
[50] Gu, G. X., Chen, C.-T., Richmond, D. J., and Buehler, M. J., 2018, "Bioinspired Hierarchical Composite Design Using Machine Learning: Simulation, Additive Manufacturing, and Experiment," Mater. Horizons, **5**(5), pp. 939–945.
[51] Gu, G. X., Chen, C.-T., and Buehler, M. J., 2018, "De Novo Composite Design Based on Machine Learning Algorithm," Extreme Mech. Lett., **18**, pp. 19–28.
[52] Mies, D., Marsden, W., and Warde, S., 2016, "Overview of Additive Manufacturing Informatics: 'A Digital Thread'," Integr. Mater. Manuf. Innov., **5**(1), pp. 114–142.
[53] Gao, W., Zhang, Y., Ramanujan, D., Ramani, K., Chen, Y., Williams, C. B., Wang, C. C. L., Shin, Y. C., Zhang, S., and Zavattieri, P. D., 2015, "The Status, Challenges, and Future of Additive Manufacturing in Engineering," Comput. Des., **69**, pp. 65–89.
[54] Thompson, M. K., Moroni, G., Vaneker, T., Fadel, G., Campbell, R. I., Gibson, I., Bernard, A., Schulz, J., Graf, P., Ahuja, B., and Martina, F., 2016, "Design for Additive Manufacturing: Trends, Opportunities, Considerations, and Constraints," CIRP Ann., **65**(2), pp. 737–760.
[55] Jee, H., and Witherell, P., 2017, "A Method for Modularity in Design Rules for Additive Manufacturing," Rapid Prototyp. J., **23**(6), pp. 1107–1118.
[56] Francois, M. M., Sun, A., King, W. E., Henson, N. J., Tourret, D., Bronkhorst, C. A., Carlson, N. N., Newman, C. K., Haut, T., Bakosi, J., Gibbs, J. W., Livescu, V.,


Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025

Vander Wiel, S. A., Clarke, A. J., Schraad, M. W., Blacker, T., Lim, H., Rodgers, T., Owen, S., Abdeljawad, F., Madison, J., Anderson, A. T., Fattebert, J.-L., Ferencz, R. M., Hodge, N. E., Khairallah, S. A., and Walton, O., 2017, "Modeling of Additive Manufacturing Processes for Metals: Challenges and Opportunities," Curr. Opin. Solid State Mater. Sci., **21**(4), pp. 198–206.
[57] Boschetto, A., and Bottini, L., 2014, "Accuracy Prediction in Fused Deposition Modeling," Int. J. Adv. Manuf. Technol., **73**(5–8), pp. 913–928.
[58] Khadilkar, A., Wang, J., and Rai, R., 2019, "Deep Learning-Based Stress Prediction for Bottom-Up SLA 3D Printing Process," Int. J. Adv. Manuf. Technol., **102**(5–8), pp. 2555–2569.
[59] Xie, W., Noble, J. A., and Zisserman, A., 2018, "Microscopy Cell Counting and Detection With Fully Convolutional Regression Networks," Comput. Methods Biomech. Biomed. Eng. Imaging Vis., **6**(3), pp. 283–292.
[60] Townsend, A., Senin, N., Blunt, L., Leach, R. K., and Taylor, J. S., 2016, "Surface Texture Metrology for Metal Additive Manufacturing: A Review," Precis. Eng., **46**, pp. 34–47.
[61] Alexander, P., Allen, S., and Dutta, D., 1998, "Part Orientation and Build Cost Determination in Layered Manufacturing," Comput. Des., **30**(5), pp. 343–356.
[62] ASTM International/U.S. Department of Defense, 2014, "ASTM A6/A6M Standard Specification for General Requirements for Rolled Structural Steel Bars, Plates, Shapes, and Sheet Piling," ASME Boiler & Pressure Vessel Code, **96**(C), pp. 1–63.
[63] ASTM, 2001, "Standard Specification for Pipe, Steel, Black and Hot-Dipped, Zinc-Coated, Welded and Seamless," p. 21.
[64] ASTM, 2014, "Standard Test Method for Tensile Properties of Plastics," ASTM D, **638**, pp. 1–17.
[65] Pilz, M., and Kamel, H. A., 1989, "Creation and Boundary Evaluation of CSG-Models," Eng. Comput., **5**(2), pp. 105–118.
[66] Zhang, Y., Bernard, A., Harik, R., and Karunakaran, K. P., 2017, "Build Orientation Optimization for Multi-Part Production in Additive Manufacturing," J. Intell. Manuf., **28**(6), pp. 1393–1407.
[67] Zhang, Y., and Bernard, A., 2013, *High Value Manufacturing: Advanced Research in Virtual and Rapid Prototyping*, CRC Press, Boca Raton, FL, pp. 411–416.
[68] Haynes, W., 2013, "Tukey's Test," *Encyclopedia of Systems Biology*, W. Dubitzky, O. Wolkenhauer, K.-H. Cho, and H. Yokota, eds., Springer, New York, NY, pp. 2303–2304.
[69] Marsan, A. L., Kumar, V., Dutta, D., and Pratt, M. J., 1998, *An Assessment of Data Requirements and Data Transfer Formats for Layered Manufacturing*, Commerce Department, National Institute of Standards and Technology (NIST), Gaithersburg, MD.
[70] Kechagias, J., and Chryssolouris, G., 1997, "Estimation of Build Times in Rapid Prototyping Processes," Proceedings of the 6th European Conference on Rapid Prototyping and Manufacturing, University of Nottingham, UK, July 1–3.
[71] Wendel, B., Rietzel, D., Kühnlein, F., Feulner, R., Hülder, G., and Schmachtenberg, E., 2008, "Additive Processing of Polymers," Macromol. Mater. Eng., **293**(10), pp. 799–809.
[72] Di Angelo, L., and Di Stefano, P., 2011, "A Neural Network-Based Build Time Estimator for Layer Manufactured Objects," Int. J. Adv. Manuf. Technol., **57**(1–4), pp. 215–224.
[73] Giannatsis, J., Dedoussis, V., and Laios, L., 2001, "A Study of the Build-Time Estimation Problem for Stereolithography Systems," Robot. Comput. Integr. Manuf., **17**(4), pp. 295–304.
[74] McClurkin, J. E., and Rosen, D. W., 1998, "Computer-Aided Build Style Decision Support for Stereolithography," Rapid Prototyp. J., **4**(1), pp. 4–13.
[75] Chollet, F., et al.et al.,2015, Keras .
[76] Abadi, M., Paul, B., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., and Zheng, X., 2016, "TensorFlow: A System for Large-Scale Machine Learning," 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16), Savannah, GA, Nov. 2–4, pp. 265–283.
[77] Kingma, D. P., and Ba, J., 2015, "Adam: A Method for Stochastic Optimization," 3rd International Conference for Learning Representations, San Diego, CA, May 7–9.
[78] Andrews, D. F., 1974, "A Robust Method for Multiple Linear Regression," Technometrics, **16**(4), p. 523.
[79] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E., 2011, "Scikit-Learn: Machine Learning in Python," J. Mach. Learn. Res., **12**, pp. 2825–2830.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111701/6578412/md_141_11_111701.pdf by Universita Di Pisa user on 24 November 2025